

CSC633 MACHINE LEARNING



TEXAS A&M UNIVERSITY

Department of Computer
Science & Engineering

CSCE 633: Machine Learning

Optimizing for Neural Networks

Texas A&M University

Outline

- Activation Function
- Backpropagation
- Optimization
- Neural Network Training and Design

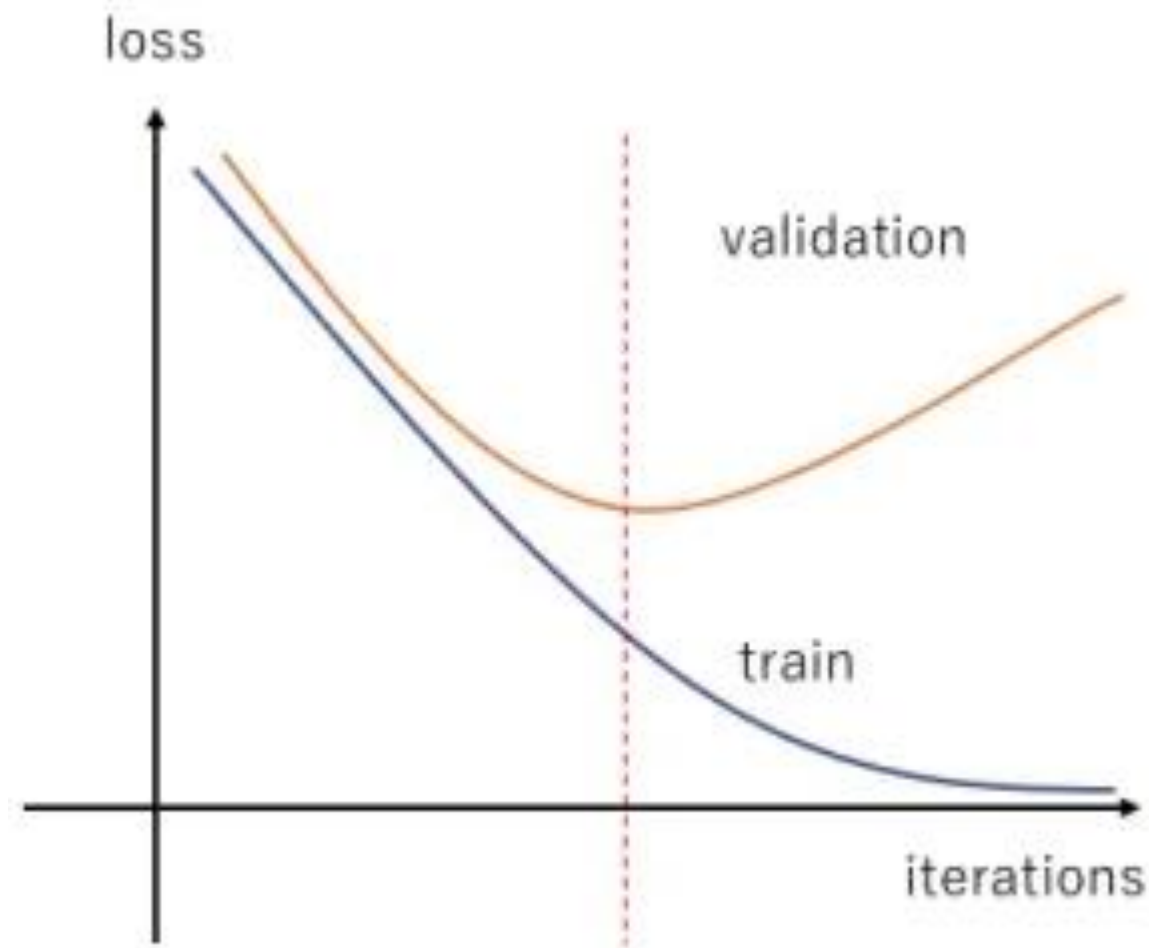
Neural Network Training (1)

Minibatch

- Potential Problem: Gradient estimates can be very noisy
- Obvious Solution: Use larger mini-batches (In theory, growingly larger)
- Advantage: Computation time per update does not depend on number of training examples.
- This allows convergence on extremely large datasets
- “Large Scale Learning with Stochastic Gradient Descent”, Leon Bottou.

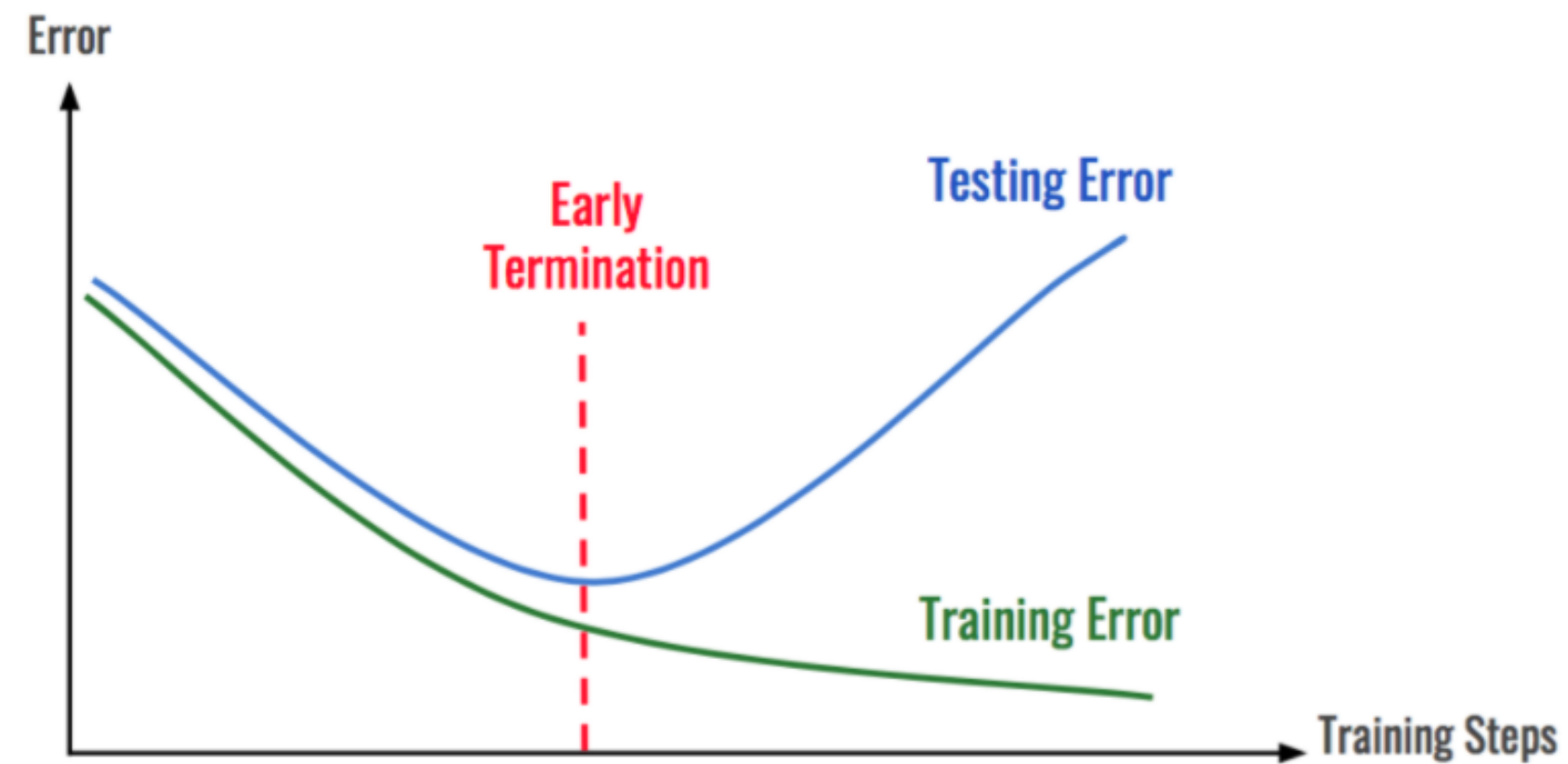
Neural Network Training (2)

Challenge: Overfitting



Neural Network Training (3)

- Early stop

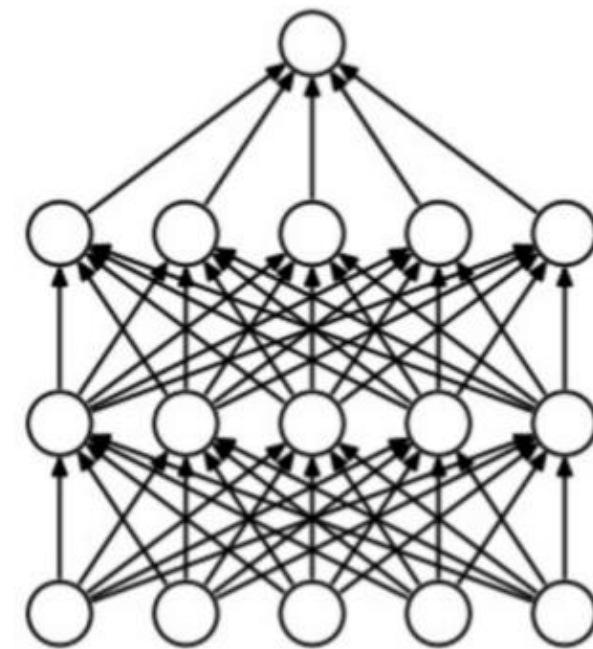


Neural Network Training (4)

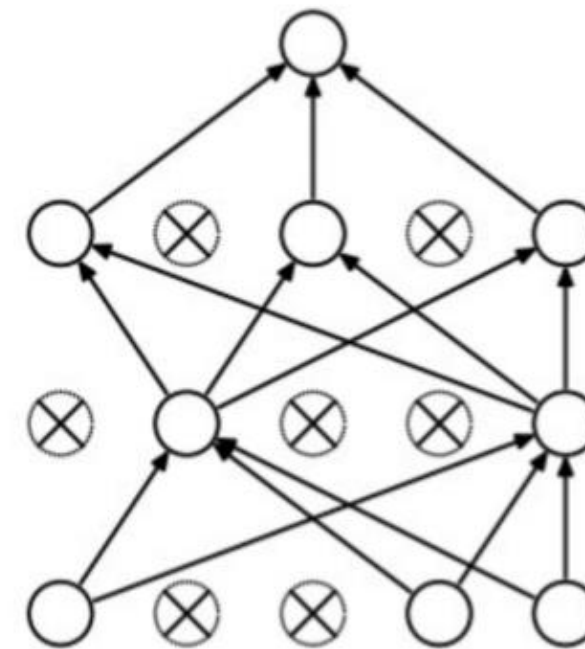
Dropout

How to avoid overfitting

- An alternative method that complements the above is **dropout**
- While training, dropout keeps a neuron active with some probability p (a hyperparameter), or sets it to zero otherwise



(a) Standard Neural Net



(b) After applying dropout.

Regularization in Deep Learning

- Norm penalty/weight decay: l_2 , l_1
- Add stochastic noise
- Data Augmentation
- Early Stopping
- Random Pruning: Dropout, Dropconnect, etc.
- Batch Normalization

Data Augmentation

Horizontal Flip



Crop

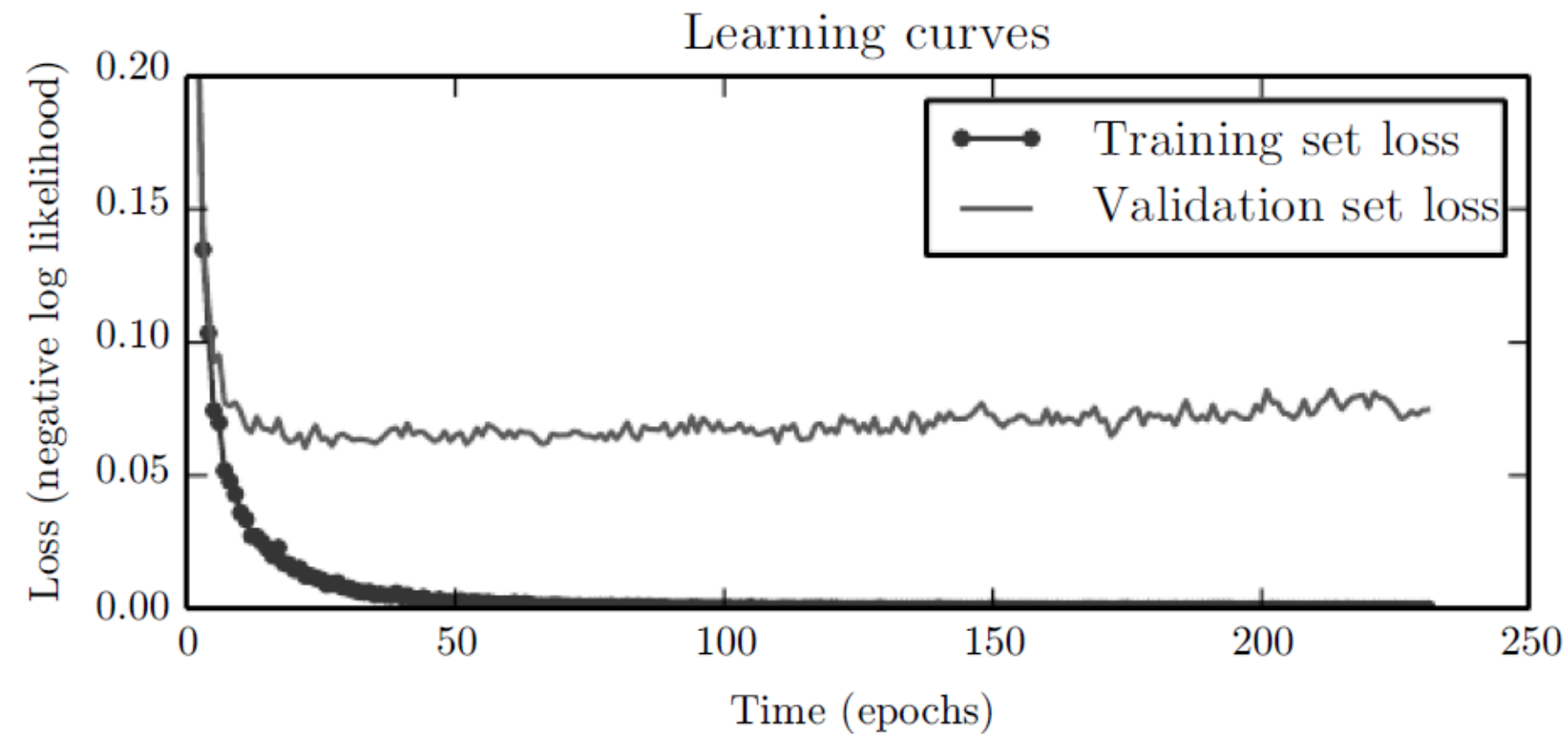


Rotate



- Adding noise to the input: a special kind of augmentation
- Be careful about the transformation applied -> **label preserving**
 - **Example:** classifying 'b' and 'd'; '6' and '9'

Early Stopping (1)



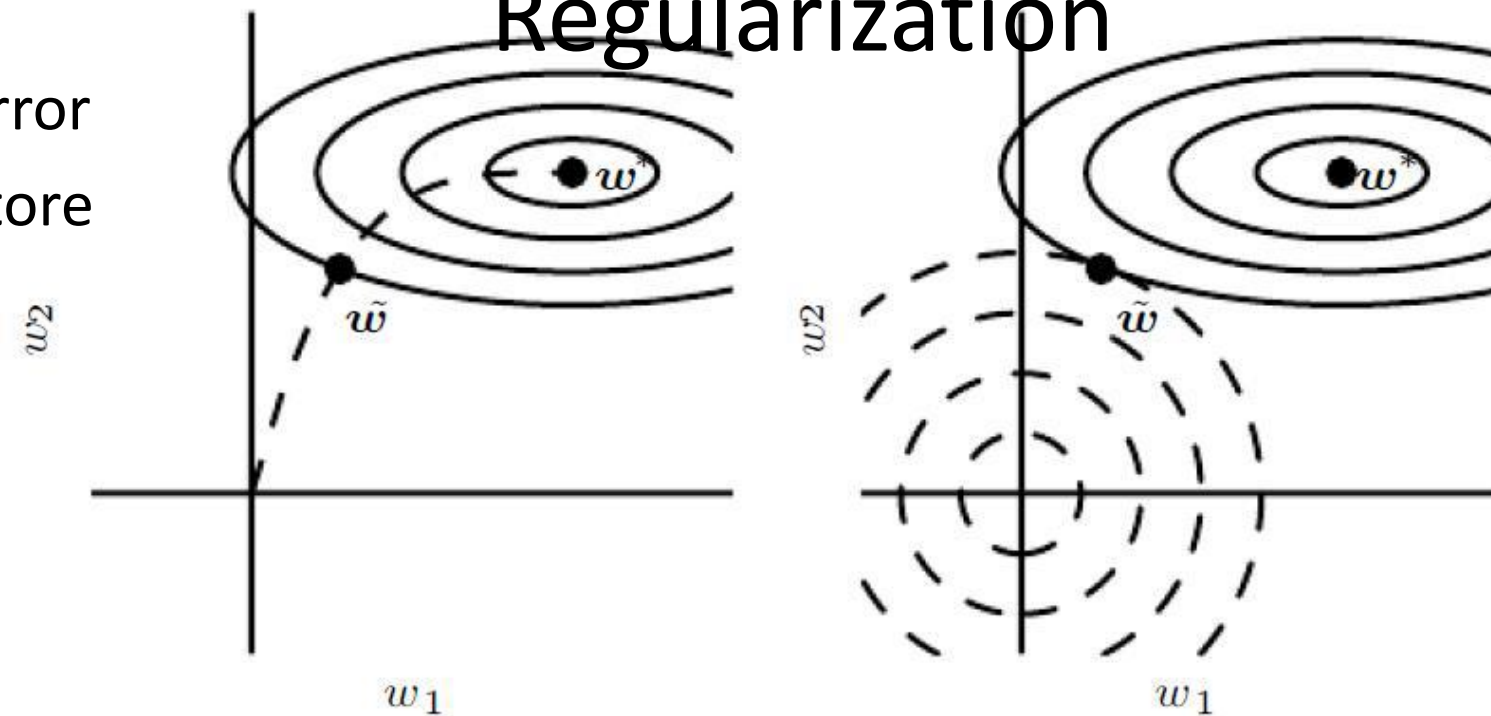
- **Idea:** don't train the network to too small training error
- Recall overfitting: Larger the hypothesis class, easier to find a hypothesis that fits the difference between the two
- Prevent overfitting: use **validation error** to decide when to stop

Early Stopping (2)

In Practice:

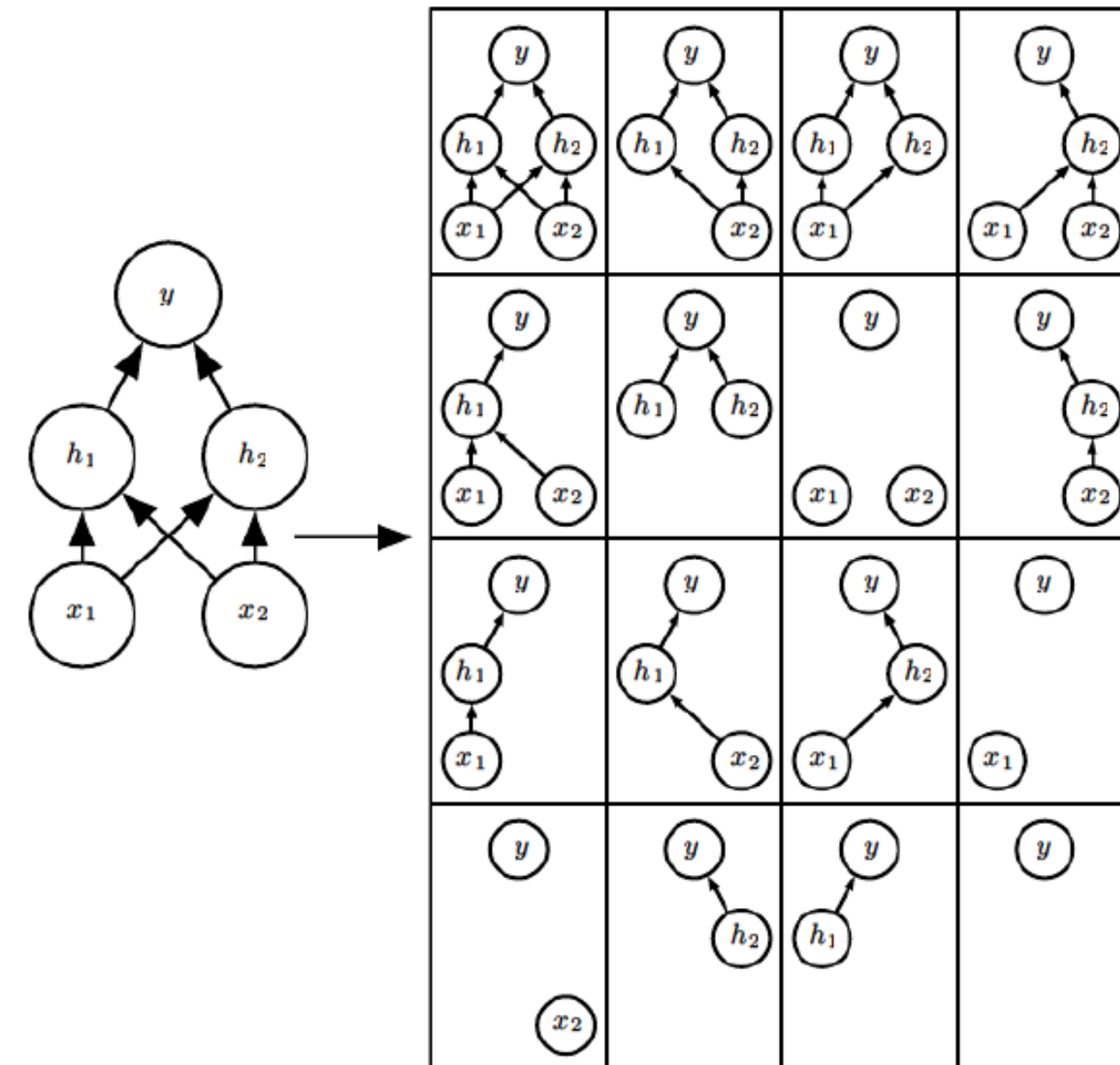
- When training, also output validation error
- Every time validation error improved, store a copy of the weights
- When validation error not improved for some time, stop
- Return the copy of the weights stored
- Training step is the hyperparameter

Early Stopping as Regularization



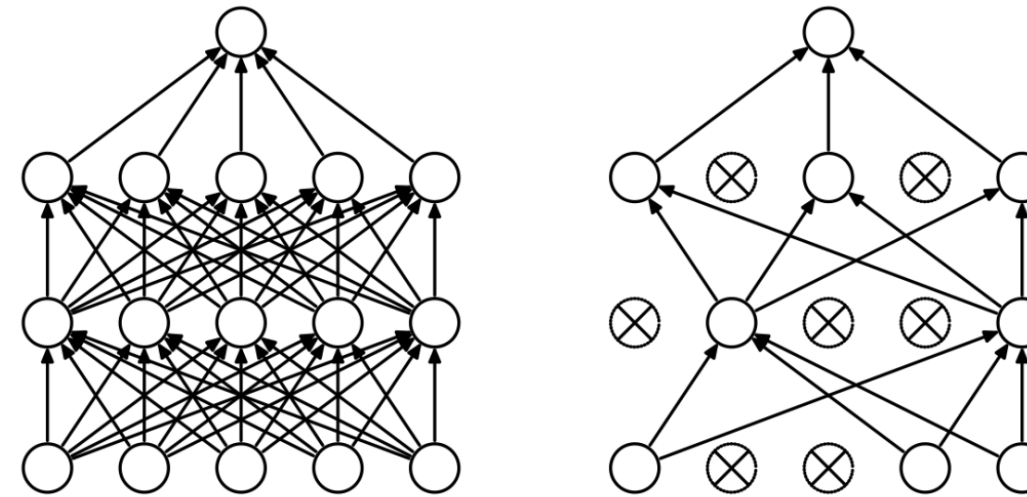
Dropout

- Randomly select weights to update
 - In each update step, randomly sample a different binary mask to all the input and hidden units
 - Multiple the mask bits with the units and do the update as usual
 - Typical dropout probability: 0.2 for input and 0.5 for hidden units
 - Very useful for FC layers, less for conv layers, not useful in RNNs



Dropout: A Stochastic Ensemble (1)

- **Dropout**: a feature-based bagging
 - Resamples input as well as **latent** features
 - With **parameter sharing** among voters



- SGD training: each time loading a minibatch, randomly sample a binary mask to apply to all input and hidden units
 - Each unit has probability α to be included (a hyperparameter)
 - Typically, 0.8 for input units and 0.5 for hidden units
- Different minibatches are used to train different parts of the NN
 - Similar to bagging, but much more efficient
 - No need to retrain unmasked units
 - Exponential number of voters

Dropout: A Stochastic Ensemble (2)

- How to vote to make a final prediction?
- Mask sampling:
 - ① Randomly sample some (typically, $10 \sim 20$) masks
 - ② For each mask, apply it to the trained NN and get a prediction
 - ③ Average the predictions
- Weigh scaling:
 - Make a single prediction using the NN with all units
 - But weights going out from a unit is multiplied by α
 - Heuristic: each unit outputs the same expected amount of weight as in training
- The better one is problem dependent

Batch Normalization (1)

- In ML, we assume future data will be drawn from same probability distribution as training data
- For a hidden layer, after training, the earlier layers have new weights and hence may generate a new distribution for the next hidden layer
- We want to reduce this internal covariate shift for the benefit of later layers

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Batch Normalization (2)

- First three steps are just like standardization of input data, but with respect to only the data in mini-batch.
- We can take derivative and incorporate the learning of last step parameters into backpropagation.
- Note last step can completely un-do previous 3 steps
- But even if so, this un-doing is driven by the **later layers**, not the **earlier layers**; later layers get to “choose” whether they want standard normal inputs or not

Neural Network Design

How to choose the number of layers and nodes

- No general rule of thumb, this depends on:
 - Amount of training data available
 - Complexity of the function that is trying to be learned
 - Number of input and output nodes
- If data is linearly separable, you don't need any hidden layers at all
- Start with one layer and hidden nodes proportional to input size
- Gradually increase

Hyperparameter Tuning

- **Learning rate:** how much to update the weight during optimization
- **Number of epochs:** number of times the entire training set pass through the neural network
- **Batch size:** the number of times the entire training set pass through the neural network
- **Activation function:** the function that introduces non-linearity to the model (e.g. sigmoid, tanh, ReLU, etc.)
- **Number of hidden layers and units**
- **Weight initialization:** Uniform distribution usually works well
- **Dropout for regularization:** probability of dropping a unit

We can perform **grid** or **randomized** search over all parameters

Challenge (1)

High memory requirements

- Memory is used to store input data, weight parameters and activations as an input propagates through the network
- Activations from a forward pass must be retained until they can be used to calculate the error gradients in the backwards pass
- Example: 50-layer neural network
 - 26 million weight parameters, 16 million activations in the forward pass
 - 168MB memory (assuming 32-bit float)

Parallelize computations with GPU (graphics processing units)

Challenge (2)

Backpropagation does not work well

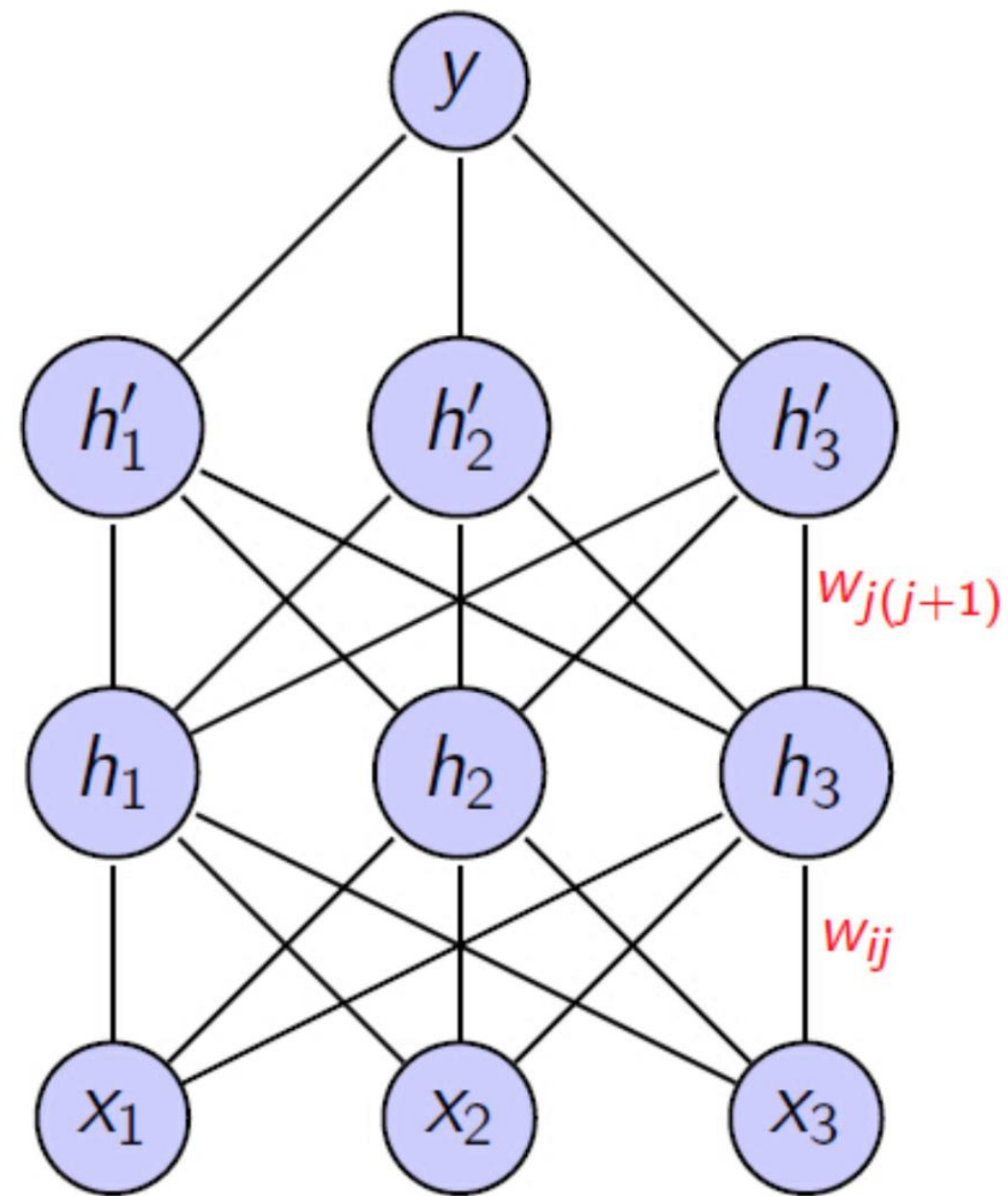
- Deep networks trained with backpropagation (without unsupervised pretraining) perform worse than shallow networks
- Gradient is progressively getting more dilute
 - Weight correction is minimal after moving back a couple of layers
- High risk of getting “stuck” to local minima
- In practice, a small portion of data is labelled

Perform pretraining to mitigate this issue

	train.	valid.	test
DBN, unsupervised pre-training	0%	1.2%	1.2%
Deep net, auto-associator pre-training	0%	1.4%	1.4%
Deep net, supervised pre-training	0%	1.7%	2.0%
Deep net, no pre-training	.004%	2.1%	2.4%
Shallow net, no pre-training	.004%	1.8%	1.9%

(Bengio et al., NIPS 2007)

Why are deep nets hard to train?



- Vanishing gradient problem in backward propagation

$$\frac{\partial \text{Loss}}{\partial w_{ij}} = \frac{\partial \text{Loss}}{\partial \text{in}_j} \frac{\partial \text{in}_j}{\partial w_{ij}} = \delta_j x_i$$

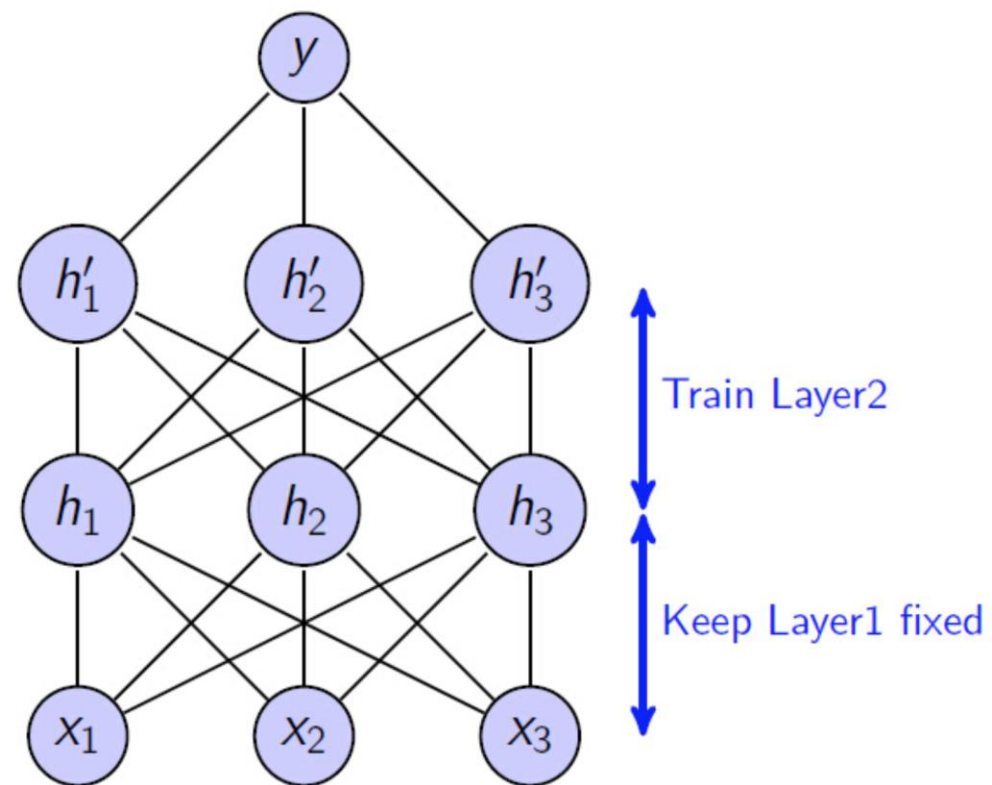
$$\delta_j = \left[\sum_{j+1} \delta_{j+1} w_{j(j+1)} \right] \sigma'(\text{in}_j)$$

δ_j may vanish after repeated multiplication

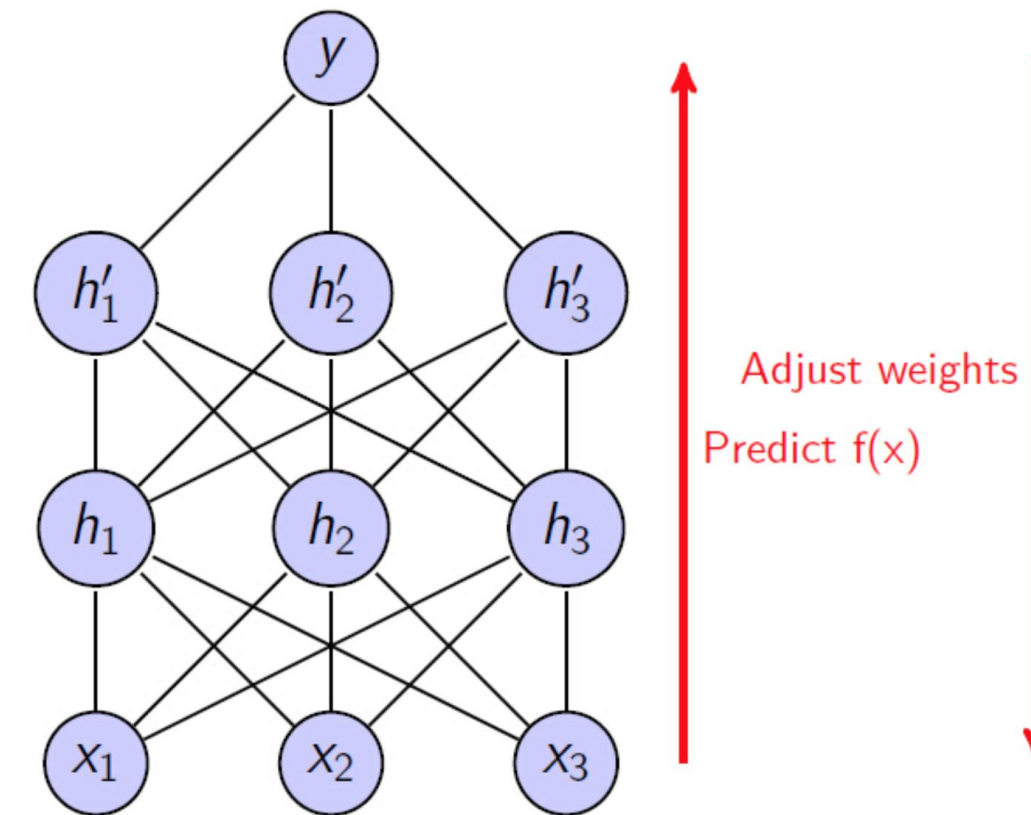
- Insufficient training data

Layer-wise Pre-training [Hinton et al. 2006]

First, train one layer at a time, optimizing data-likelihood objective $P(x)$



Finally, fine-tune labeled objective $P(y|x)$ by Backpropagation

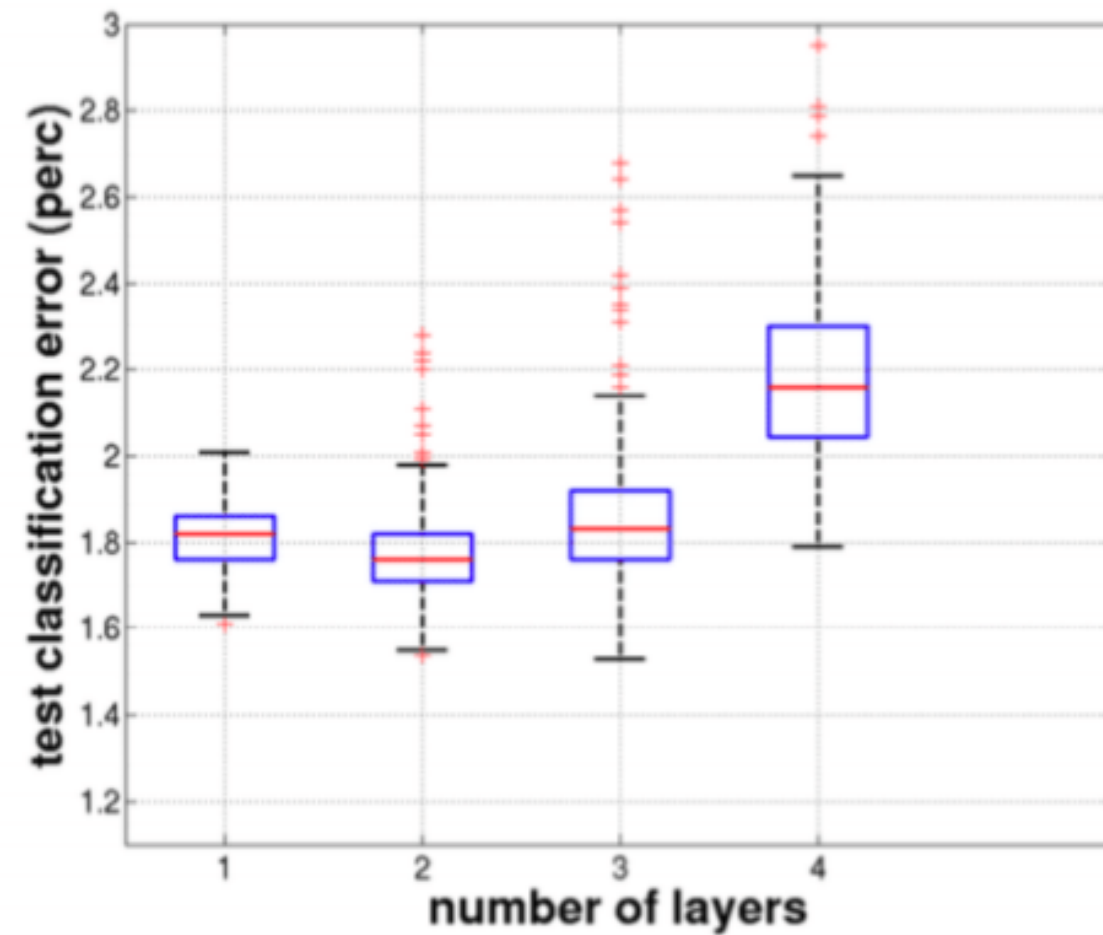


Unsupervised Pretraining (1)

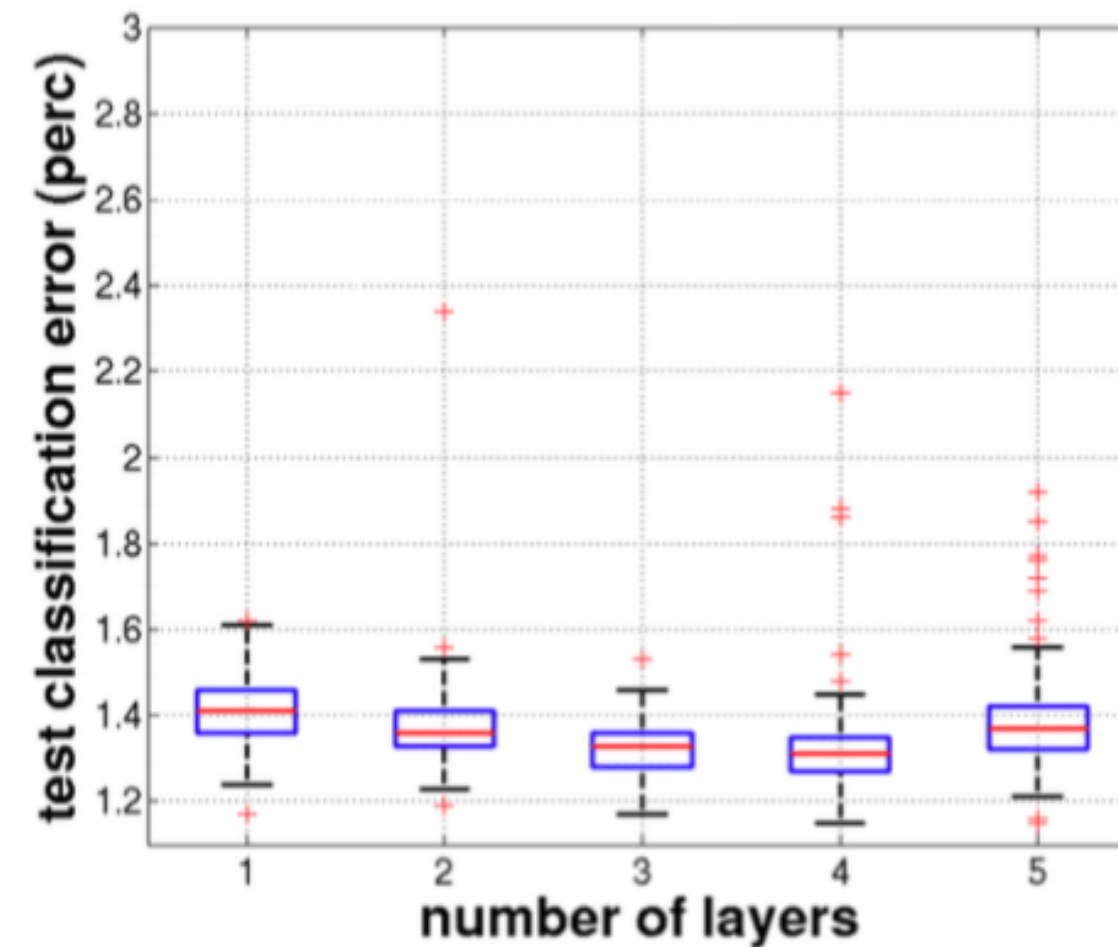
- This idea came into play when research studies found that a DNN trained on a particular task (e.g. object recognition) can be applied on another domain (e.g. object subcategorization) giving state-of-the-art results
- **1st part: Greedy layer-wise unsupervised pre-training**
 - Each layer is pre-trained with an unsupervised learning algorithm
 - Learning a nonlinear transformation that captures the main variations in its input (the output of the previous layer)
- **2nd part: Supervised fine-tuning**
 - The deep architecture is fine-tuned with respect to a supervised training criterion with gradient-based optimization
- We will examine the **deep belief networks** and **stacked autoencoders**

Unusual form of regularization: minimizing variance and introducing bias towards configurations of the parameter space that are useful for unsupervised learning

Unsupervised Pretraining (2)



Without pre-training



With pre-training

[Source: Erhan et al., 2010]

Unsupervised Pretraining (3)

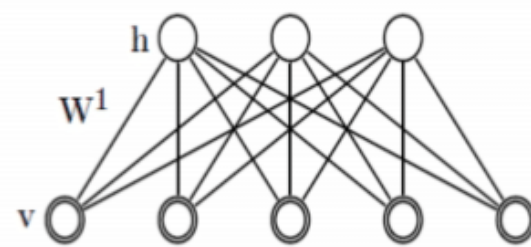
Deep Belief Networks (Hinton et al. 2006)

- Pretraining is implemented by stacking several layers of Restricted Boltzmann Machines (RBM) in a greedy manner
- Assuming joint distribution between hidden h_i and observed variables x_j with parameters $\mathbf{W}, \mathbf{b}, \mathbf{c}$
$$P(\mathbf{x}, \mathbf{h}) \propto \exp(\mathbf{h}^T \mathbf{W} \mathbf{x} + \mathbf{b}^T \mathbf{x} + \mathbf{c}^T \mathbf{h})$$
$$P(\mathbf{x}|\mathbf{h}) = \prod_j P(x_j|\mathbf{h}), P(x_j = 1|\mathbf{h}) = \text{sigmoid}(b_j + \sum_i W_{ij} h_i)$$
$$P(\mathbf{h}|\mathbf{x}) = \prod_i P(h_i|\mathbf{x}), P(h_i = 1|\mathbf{x}) = \text{sigmoid}(c_i + \sum_j W_{ij} x_j)$$
- RBM trained by approximate stochastic gradient descent
- This representation is extended to all hidden layers
- The RBM parameters correspond to the parameters of the feed-forward multi-layer neural network

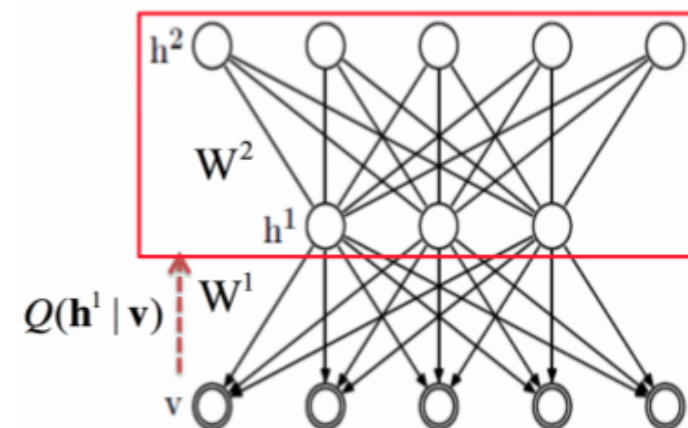
Unsupervised Pretraining (4)

Deep Belief Networks (Hinton et al. 2006)

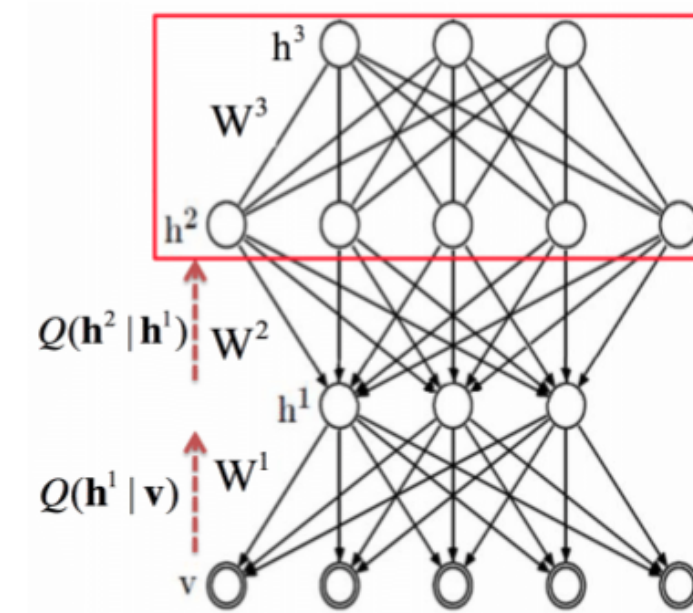
- Step 1: Construct an RBM with an input and hidden layer and train to find $\mathbf{W}^{(1)}$
- Step 2: Stack another hidden layer on top of the RBM to form a new RBM
 - Fix $\mathbf{W}^1$. Assume $\mathbf{h}^{(1)}$ as input. Train to find $\mathbf{W}^{(2)}$.
- Step 3: Continue to stack layers and find weights $\mathbf{W}^{(3)}$, etc.



Step 1



Step 2



Step 3

Monitor Your Training Curve

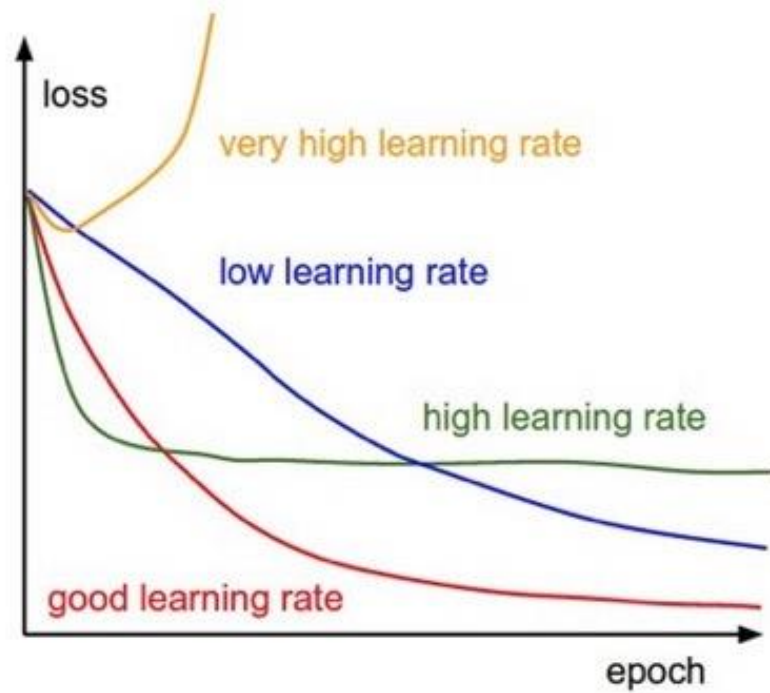


Figure 1

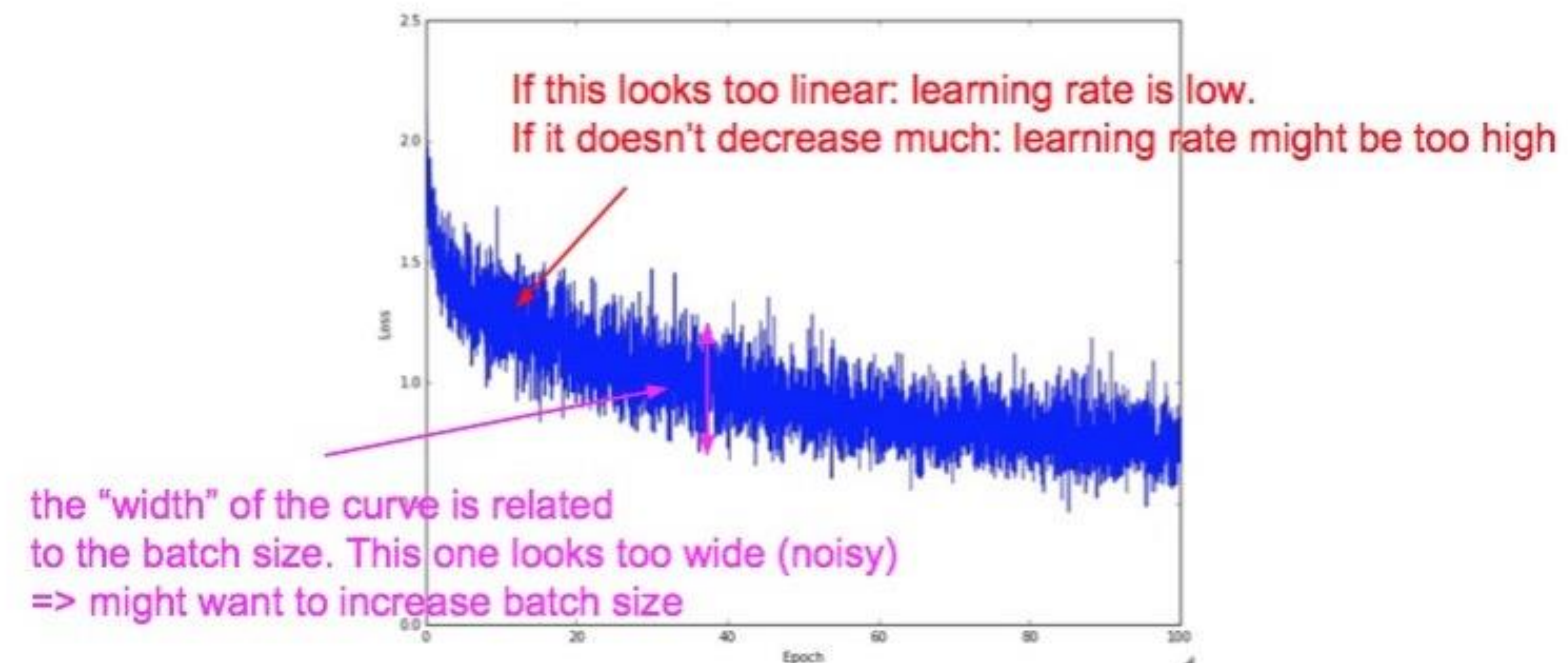


Figure 2

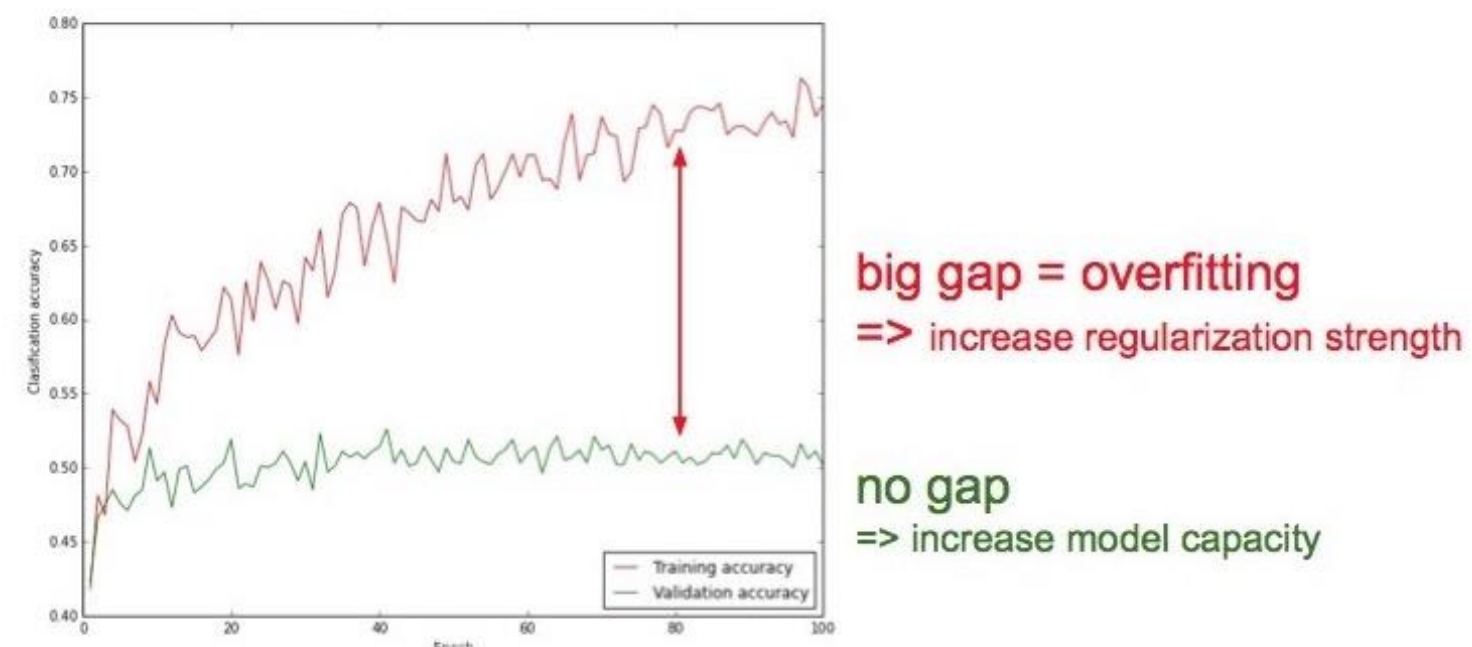
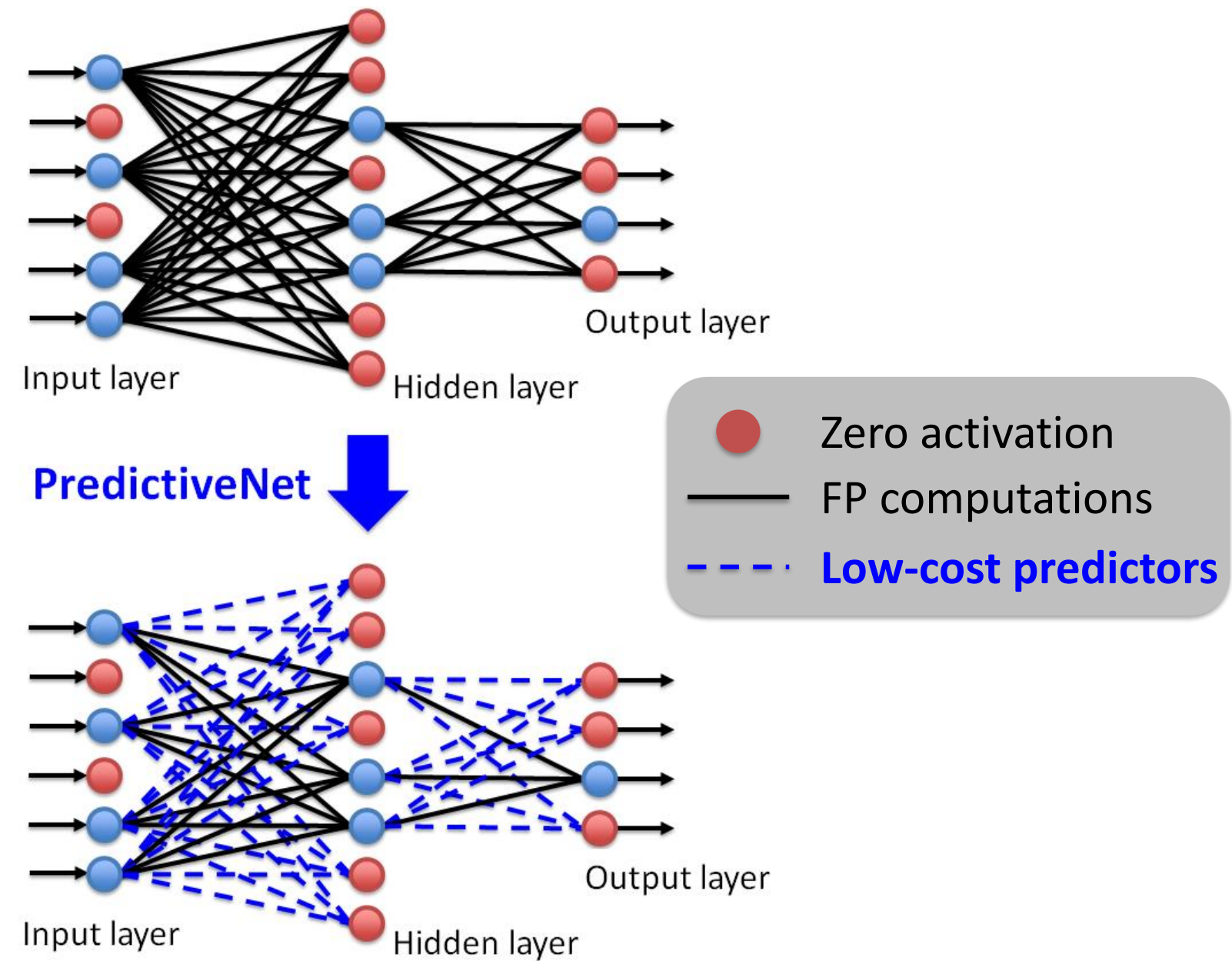


Figure 3

Predict Zero Activations with Low Cost?



Takeaways

- Deep Neural Networks
- How to train deep neural networks
- Next Time: Examples of Important Deep Neural Network Classifiers